Beyond the Agent Object: Entity Component Systems as an Architecture for Agent-Based Modeling

Franz Scharnreitner BSc.

Agent-based models are commonly implemented around agent objects that combine identity, state, and behavior. Although this organization mirrors the intuitive description of autonomous agents, it makes predefined agent types the principal unit of model construction and can obscure the population-level processes through which agents interact. This paper examines Entity Component Systems (ECS) as an alternative architecture for agent-based modeling. ECS represents agents as dynamically composed sets of state components and behavior as systems operating on all entities with the required components. We develop three connected arguments: ECS supports structural rather than merely parametric heterogeneity; it aligns executable model structure with population-level processes; and it exposes the data dependencies required for simultaneous and parallel interaction. These arguments are developed through an ECS reconstruction and extension of Hodgson and Knudsen's traffic-convention model and a comparison with an agent-centered implementation. A secondary engineering evaluation considers cache locality, multicore scaling, and the prospects for GPU execution. The paper positions ECS not only as a computational optimization, but as an alternative way of specifying agent-based models.

July 25, 2026

Contents

Status of this document

This is a manuscript scaffold. Each section records the claim it should establish, the evidence it requires, and important qualifications. Replace these notes with finished prose only after the corresponding implementation or analysis exists.

1. Introduction

Consider Hodgson and Knudsen's model of traffic-convention formation. Drivers moving in opposite directions on a two-lane ring choose a side using only a bounded view ahead. Their choices combine responses to traffic moving in the same and opposite directions, near-field collision avoidance, and an age-dependent habit toward one side. Collisions remove drivers and thereby select among their fixed dispositions, while surviving drivers reinforce the side on which they have travelled; repeated local decisions can consequently stabilize a population-wide left- or right-driving convention [1]. Our extension makes same-direction response, opposite-direction response, avoidance, habit formation, local convention perception, and imitation through successful-driver traces independently composable capabilities. Drivers may possess any combination of them, and evolutionary replacement can change the distribution of those combinations within the population. Because these capabilities overlap, they do not form a natural taxonomy of driver types. This modeling choice exposes a broader architectural question: should an agent be implemented as an object that owns its state and behavior, or as a composition of capabilities participating in population-level processes?

ABMs explain macro-level regularities as the emergent result of heterogeneous agents and their interactions. In most implementations, the scientific agent is mapped onto a software object that combines identity, state, and behavior. This mapping is intuitive, but it is not neutral: it encourages modelers to begin with a taxonomy of agent types, locate behavior in agent-local step functions, and treat population-level processes as consequences of repeatedly invoking those functions.

Entity–component–systems offer a different starting point. Entities supply identity, components represent state and capabilities, and systems implement transformations over every entity possessing a required component signature. The resulting architecture is organized around composition and processes rather than class membership and agent-local methods.

This paper investigates whether that architectural change is useful for scientific ABMs and how it manifests in concrete modeling choices and practices.

1.1. What Makes an Agent?

What, exactly, is an agent? In a computational model, it may be a person, a household, a firm, a bank, a government—or even a car. What these otherwise disparate entities share is that the model attributes state and actions to them: they consume, produce, lend, tax, or move. How those actions are represented, however, varies across modeling traditions. In standard CGE models, agents are typically aggregate categories whose behavior is encoded in systems of demand, supply, and equilibrium equations. Microsimulation models represent persons or households individually but commonly process them using shared accounting or behavioral rules, often without direct interaction. Agent-based models go further by representing agents as individual computational entities whose states evolve through their actions and interactions.

This explicit representation makes ABMs particularly well suited to modeling heterogeneous populations. In practice, however, heterogeneity is often encoded either parametrically, through different values of a shared set of attributes, or categorically, through predefined agent

types. In the following sections, we distinguish a third form, which we call structural heterogeneity: agents may differ in the state variables and capabilities they possess, independently of any fixed type hierarchy. An entity–component–system architecture supports this form of heterogeneity by constructing agents through the composition of components.

Interactions are rarely the behavior of one agent alone. They are relational processes through which agents jointly produce population-level dynamics and, potentially, nonlinear feedback. Yet many ABM frameworks organize execution around methods invoked on one agent at a time. This can make a process involving several agents appear to belong to a single participant, obscuring who participates and when the resulting state changes take effect. The distinction becomes particularly important for simultaneous interactions, which require separate phases for observation, intention formation, conflict resolution, and commitment.

These semantic issues are closely connected to parallel execution. Specialized frameworks such as FLAME GPU demonstrate that agent-centered models can be executed efficiently in parallel: agents of the same type and state execute agent functions concurrently on the GPU. Safe interaction is achieved through explicit messages, while layers or dependency graphs determine the order in which functions execute. Parallelism therefore remains possible, but requires a comparatively constrained execution model in which agent types, states, messages, and dependencies must be specified explicitly. (FLAME GPU documentation (https://docs.flamegpu.com/guide/creating-a-model/index.html°))

ECS offers a different organizing principle. Even in a structurally heterogeneous population, systems apply homogeneous operations to agents that share particular components. Component-oriented storage can make these operations amenable to batching, while systems expose their data dependencies as a basis for safe scheduling and parallel execution. ECS does not make parallelism automatic, but aligns the unit of execution with population-level processes rather than individual agent methods.

1.2. What Changes with ECS?

We examine ECS as both a modeling abstraction and an execution architecture for ABMs. The investigation is organized around four questions:

1. **RQ1 — Compositional heterogeneity:** How does ECS represent agents with overlapping and dynamically changing component signatures that encode behavioral capabilities, compared with parametric and type-based representations?
2. **RQ2 — System-centered modeling:** What is gained and lost when behavior is expressed as systems operating on selected populations rather than as methods owned by individual agents?
3. **RQ3 — Interaction semantics:** How can system dependencies express the observation, intention, resolution, and commitment stages of simultaneous interactions, and what opportunities do they provide for parallel execution?
4. **RQ4 — Execution:** Under which workloads do component-oriented data layouts improve cache locality and multicore scaling, and what constraints do they impose on GPU execution?

Our contribution is threefold. Conceptually, we distinguish parametric, type-based, and compositional heterogeneity and map the principal concepts of ABM to entities, components, queries, systems, and resources. Methodologically, we reconstruct a classical economic ABM in both agent-centered and ECS architectures, extend it with overlapping and dynamically changing component signatures that encode behavioral capabilities, and formulate simultaneous interaction as a staged process derived from system dependencies. Empirically, we compare the architectures across relevant workloads, measuring cache behavior and multicore scaling while assessing their suitability for GPU execution.

2. Agent-Centered Architectures and Their Commitments

2.1. The agent object convention

A common implementation of an ABM assigns each scientific agent a corresponding software object. The object has an identity and a named type or record structure, while its fields hold both externally visible state, such as position or wealth, and internal state, such as beliefs, preferences, or memory. Behavior is associated with that representation through methods selected by inheritance, dispatch, or explicit calls. We call this organization the **agent object convention**. Here, "object" is used in the broad architectural sense of a state-bearing software representation; it does not require a particular object-oriented language or an inheritance hierarchy.

Execution under this convention is typically organized by a scheduler. At each tick or event, the scheduler selects an agent and invokes a step function or event handler associated with its representation. During that invocation, the agent may inspect its own fields, query nearby agents or shared model state, choose an action, and mutate itself, another agent, or the environment. The population-level transition is consequently assembled from repeated invocations of agent-level behavior. Toolkits differ substantially in how they order these invocations: they may use fixed or randomized activation, simultaneous-update buffers, event queues, multiple agent types, or user-defined schedules [2].

This organization makes the agent object the principal unit of both description and execution. The state belonging to an agent is determined by its record or type, behavior is approached by asking what that agent does during its activation, and an interaction commonly enters the implementation through one participating agent's method. These choices are architectural commitments rather than requirements of agent-based modeling itself. The same scientific model could instead store state separately from identity and express behavior as transformations over every agent satisfying particular conditions.

The comparison in this paper is therefore between **agent-centered** and **system-centered** architectures, not between Julia and another language, nor between object-oriented and non-object-oriented programming. Agent-centered frameworks can support composition, custom activation, event-based execution, buffered updates, and efficient internal storage; an ECS implementation can likewise expose object-like interfaces. The distinction concerns where the executable model locates state and behavior. Our agent-centered reference is consequently intended to be idiomatic rather than a deliberately weak inheritance-based baseline.

2.2. Three kinds of heterogeneity

Parametric, type-based, and compositional heterogeneity are distinct but not mutually exclusive dimensions along which agents may differ. A model may combine any or all of them. Let I_t be the finite, nonempty set of agents present at time t and let $N_t = |I_t|$. This notation permits entry, exit, and replacement; neither population size nor agent identifiers need remain fixed. The traffic model provides the running example: drivers have a travel direction, differ in component composition, and carry component-specific sensitivity values.

For every $i \in I_t$, let $\Theta_t(i)$ denote the agent's current parameter, i.e

$$\Theta_t : I_t \to \mathbf{\Theta} \tag{1}$$

is the map that assigns every element a parameter space of the universe $\mathbf{\Theta}$ space and let $\theta_t(i) \in \Theta_t(i)$ denote its parameter vector. The space $\Theta_t(i)$ is indexed by time and need not remain fixed. For any nonempty group $J \subseteq I_t$ whose members share a current parameter space, $\Theta_t(i) = \Theta$ for every $i \in J$, the population exhibits **parametric heterogeneity** within J if $\theta_t(i) \neq \theta_t(j)$ for some $i, j \in J$. Drivers with the same component signature but different sensitivity coefficients provide an example.

A population exhibits **type-based heterogeneity** when a nominal classification $\tau_t : I_t \to \mathcal{T}$ assigns every agent to one member of a predefined, exhaustive type set $\mathcal{T}$, and $\tau_t(i) \neq \tau_t(j)$ for some $i, j \in I_t$. The classification induces the pairwise-disjoint partition $I_t = \cup_{\tau \in \mathcal{T}} I_t^\tau$, where $I_t^\tau = \{i \in I_t \mid \tau_t(i) = \tau\}$. A type may determine an admissible state space or transition law, but parameter values and component signatures may still vary within it. For example, the traffic model's two directions could be encoded as the nominal types `ClockwiseDriver` and `CounterclockwiseDriver`. The case-study implementations instead store direction without introducing nominal driver subtypes; the counterfactual illustrates that type-based heterogeneity is a property of the model's nominal classification, not of a particular programming language or an inheritance hierarchy.

A population exhibits **compositional heterogeneity** when agents differ in the sets of components that constitute their modeled structure. Let $\mathcal{U}$ be the universe of agent component types and fix the scientifically relevant component types $c_1, ..., c_m \in \mathcal{U}$. Define the full component-signature function and its structural projection by

$$\kappa_t : I_t \to \mathcal{P}(\mathcal{U}), \quad \kappa_t^{\mathrm{str}}(i) = \kappa_t(i) \cap \{c_1, ..., c_m\}. \tag{2}$$

Thus $\kappa_t(i)$ is the complete implementation signature, whereas $\kappa_t^{\mathrm{str}}(i)$ is the signature used to analyze compositional heterogeneity. Mandatory components may occur among $c_1, ..., c_m$ but cannot by themselves generate heterogeneity. Transient buffers used only to implement a schedule remain in $\kappa_t(i)$ so systems can query them, but are omitted from that list unless their presence has a substantive interpretation.

At this level, a component is only a tag: membership of c in $\kappa_t(i)$ records that component c is present. Section 2.2 assigns neither a data domain nor a parameter block to that tag. This keeps the general definition of compositional heterogeneity independent of the ECS storage representation introduced in Section 3.

For $k \in \{1, ..., m\}$, define the component-incidence indicator

$$\chi_{ik}(t) = \begin{cases} 1 \text{ if } \ c_k \in \kappa_t^{\mathrm{str}}(i) \\ 0 \text{ otherwise} \end{cases}, \tag{3}$$

and collect the indicators in

$$\chi_i(t) = (\chi_{i1}(t), ..., \chi_{im}(t)) \in \{0, 1\}^m. \tag{4}$$

After fixing any ordering of I_t, the incidence matrix $X_t = (\chi_{ik}(t)) \in \{0,1\}^{N_t \times m}$ describes the population's component composition. For $x \in \{0,1\}^m$, the empirical distribution of structural signatures is

$$\pi_t(x) = \frac{1}{N_t} |\{i \in I_t \mid \chi_i(t) = x\}|. \tag{5}$$

The population is compositionally heterogeneous precisely when $\kappa_t^{\mathrm{str}}(i) \neq \kappa_t^{\mathrm{str}}(j)$ for some $i, j \in I_t$, or equivalently when $|\{x \in \{0,1\}^m \mid \pi_t(x) > 0\}| > 1$.

The three dimensions can be described jointly by associating agent i with

$$(\tau_t(i), \kappa_t(i), \Theta_t(i), \theta_t(i)), \quad \theta_t(i) \in \Theta_t(i). \tag{6}$$

Compositional heterogeneity is **dynamic** if a persistent agent can satisfy $\kappa_{t+1}^{\mathrm{str}}(i) \neq \kappa_t^{\mathrm{str}}(i)$, or if entry, exit, and replacement change the population distribution so that $\pi_{t+1} \neq \pi_t$. Heterogeneity itself is not an ECS innovation. The claim here is that ECS makes component incidence and changes to it explicit without requiring an exhaustive taxonomy of combination-specific agent classes.

2.3. Behavior located in agents

The preceding forms of heterogeneity describe what may differ between agents; they do not yet specify where the corresponding behavior is implemented. With the notation of Section 2.2, the relevant descriptor of agent i is

$$a_t(i) = (\tau_t(i), \kappa_t(i), \Theta_t(i), \theta_t(i)), \quad \theta_t(i) \in \Theta_t(i). \tag{7}$$

Let W denote a complete world state containing these agent descriptors, all remaining model state, and the environment, and let $N_i(W)$ be the information about that state made available to agent i. For an arbitrary current state W, write $\tau_W(i)$, $\kappa_W(i)$, and $\theta_W(i)$ for the corresponding entries of agent i's descriptor in that state; at $W = W_t$ they agree with the time-indexed quantities above. In an agent-centered architecture, activating i invokes an agent-level transition operator

$$F_i(W) = f_{\tau_W(i)}(\kappa_W(i), \theta_W(i), N_i(W), W). \tag{8}$$

The right-hand side is understood to return an updated world state. The type $\tau_W(i)$ may select the implementation by dispatch, $\kappa_W(i)$ may select branches or delegated behaviors associated with available components, and $\theta_W(i)$ supplies the agent-level parameter values used by those behaviors. Any additional agent state is already part of W. The operator may update the agent, other agents, or the environment; it may also replace $\kappa_W(i)$ and $\theta_W(i)$ with a new signature and a value in the corresponding parameter space. Agent-centered organization therefore does not preclude compositional or dynamically changing heterogeneity. Its defining feature is that these transformations are reached through the activation of a particular agent representation.

A scheduler turns the individual operators into a population transition. For a tick with $N_t = |I_t|$ scheduled agents, let $\sigma_t : \{1, \ldots, N_t\} \to I_t$ give their activation order. In the simplest sequential case,

$$W_t^0 = W_t, \quad W_t^r = F_{\sigma_t(r)}(W_t^{r-1}), \quad W_{t+1} = W_t^{N_t}. \tag{9}$$

Agent $\sigma_t(r)$ consequently observes $N_{\sigma_t(r)}(W_t^{r-1})$: an earlier activation may change the state seen by a later one. Activation order is behaviorally relevant whenever two operators do not commute,

$$F_i \circ F_j \neq F_j \circ F_i. \tag{10}$$

This formulation also shows why a single agent step can obscure timing. One invocation may successively perform perception, choice, action, interaction resolution, and learning even though the scientific model assigns those processes different information sets or commitment times. Reading and writing the world during the same invocation gives each process the intermediate state created by the preceding code unless the implementation introduces explicit buffers or phases.

Sequential semantics are not required by the agent-centered architecture. A framework can first invoke an intention function for every agent from the same state,

$$p_i = G_{i,t}(W_t), \tag{11}$$

and then resolve and commit the collection $(p_i)_{i \in I_t}$ in a separate model-level operation. The traffic reference implementation demonstrates both possibilities with the same `Car` record: its sequential schedule calculates a driver's LR value and moves that driver before activating the next, whereas its simultaneous contrast calculates all LR values from a frozen state before committing movement. The architectural question is thus not whether an agent-centered model **can** express simultaneous behavior, but whether agent-local activation is the clearest primary unit for specifying processes that operate over overlapping populations.

2.3.1. Behavior located in practice

The major general-purpose toolkits separate the code that defines a behavior from the mechanism that activates it, but they commonly retain an individual agent as the unit of activation. Mesa illustrates the object-oriented form of this convention. Behavior is ordinarily written as methods of a Python `Agent` subclass, while a model-level step selects an `AgentSet` and invokes a named method through operations such as `do` or `shuffle_do`. The latter makes the population traversal explicit and permits filtering, grouping, fixed order, or randomized order, yet each invocation still enters the model through one agent object. If its method immediately mutates the model, that traversal implements a composition of F_i operators under the chosen σ_t. Nothing prevents a Mesa model from invoking several methods in successive passes, storing proposals, or performing resolution in `Model.step`; indeed, the `AgentSet` interface makes such staging natural. The architectural point is therefore that the idiomatic lexical home of behavior is an agent method, not that Mesa lacks population-level control or alternative activation regimes [3].

MASON gives the scheduler an even more independent status. Executable objects implement the Java `Steppable` interface and receive the shared `SimState` when their `step` method is called. Agents often implement `Steppable` themselves, so state and behavior remain co-located, but any helper, environmental process, or model-level coordinator may be scheduled in exactly the

same way. Its priority queue can place one-shot or repeating events at real-valued times, order multiple phases at the same time, and wrap collections in fixed or randomized sequences. Thus MASON can encode an asynchronous event model directly, or a staged synchronous model by scheduling observation, resolution, and commitment as distinct `Steppable`s. This flexibility weakens any claim that agent-centered frameworks require one undifferentiated tick; what persists in typical models is the convention that an agent's `Steppable.step` supplies its F_i [4].

NetLogo distinguishes lexical location from activation especially clearly. Its behavior is written in named procedures rather than methods stored inside turtle, patch, or link objects. Nevertheless, `ask` executes a command block in the context of each member of an agentset, so the active `self`, its owned variables, and its local neighborhood organize the computation. For an agentset, ordinary `ask` visits agents serially in randomized order and commits changes as it goes. Consequently, noncommuting actions can expose the same $F_i \bigcirc F_j \neq F_j \bigcirc F_i$ dependence described above. The observer's `go` procedure can instead issue several `ask` passes and use temporary state to separate decision from commitment. NetLogo also retains `ask-concurrent`, which simulates turn-taking concurrency, but this is neither a general simultaneous-update guarantee nor the recommended basis for new models. The language therefore supports explicitly staged population processes while making agent-context commands the characteristic idiom [5].

Agents.jl reaches a similar organization without object-oriented lexical ownership. A discrete-time `StandardABM` is normally constructed with an external Julia function `agent_step!(agent, model)` and, optionally, a `model_step!(model)` function. A scheduler chooses which agents receive the former and in what order. Behavior is thus textually a free function and may use multiple dispatch, yet its ordinary activation still has the shape of $F_{i(W)}$. Model steps, custom schedulers, buffered fields, and repeated population passes can implement model-wide or simultaneous phases, while `EventQueueABM` provides continuous-time event execution instead of a once-per-tick sweep. Agents.jl is therefore appropriately described as agent-centered, not as class-bound: it deliberately exposes both agent-level and model-level transition hooks [6].

FLAME GPU provides a useful limiting contrast. Behavior is lexically defined in external agent functions, but a function associated with an agent type and state is launched as a GPU kernel across the eligible population. Messages separate communication phases, and ordered layers or a dependency graph determine when kernels and host-level functions execute; functions within a valid layer may execute concurrently. Activation is consequently already population-wide and explicitly phased, even though eligibility and behavioral association remain organized by agent type and state rather than by a query over independently composed capabilities [7].

Across these frameworks, then, "behavior located in agents" is a statement about the default route by which a transition is specified and reached, not a restriction on expressive power. All can express model-level processes, buffers, multiple phases, and custom timing. What changes among sequential, staged, simultaneous, and event-based formulations is how the framework constructs the population transition from the F_i: in particular, whether σ_t exposes intermediate worlds or whether proposals are computed from a common W_t before commitment. The system-centered ECS alternative developed next changes the default unit of specification. Rather than begin with an agent activation and recover a population process from repeated

calls, it begins with a process whose query selects every entity possessing the required state, making participation and process timing explicit in the executable structure.

2.4. From agent types to parameter spaces: The limitations of traditional agent based frameworks

Many traditional ABM frameworks make the agent's declared kind the default place in which its admissible fields are specified. In the notation of Section 2.2, this common design can be represented by a schema map $\tilde{\Theta} : \mathcal{T} \to \mathbf{\Theta}$ such that $\Theta_{t(i)} = \tilde{\Theta}\left(\tau_{t(i)}\right)$. The map need not be surjective: several types may share a schema, and some admissible spaces in $\mathbf{\Theta}$ may be unused. Nor is this relationship necessary. Dynamic fields, delegated objects, flags, traits, and model-level tables can make the effective parameter space depend on more than the nominal type.

Where both the software type and its declared schema remain fixed over an agent's lifetime, one may write $\tau_{t(i)} = \tau(i)$ and $\Theta_{t(i)} = \Theta(i)$. This is a frequent framework default, not a universal property of agent-centered modeling.

2.4.1. Type based heterogeneity in practice

The major general-purpose toolkits make nominal agent kinds and their fields the most visible route to heterogeneity, but they do so through different language mechanisms. Reviews therefore commonly classify toolkit support in terms of agent classes or breeds, attributes, and scheduling facilities [2]. Relative to the distinctions in Section 2.2, these mechanisms usually give τ and Θ a native representation: a declared kind identifies a schema and each instance supplies its values θ. By contrast, a scientifically interpreted component signature κ is usually a convention constructed by the modeler rather than a first-class object on which the framework's storage and execution are based. This is a claim about the affordances made explicit by each toolkit, not a limit on what can be programmed in a general-purpose language.

Mesa follows Python's class-based idiom. Modelers typically subclass `Agent`, declare or initialize instance attributes, and introduce further subclasses for behaviorally distinct populations [3]. A class label can therefore realize τ, its expected attributes define an admissible Θ, and their per-agent contents are θ. Python does not, however, force a closed schema: mixins, delegated behavior objects, dynamically attached attributes, and boolean or enumerated capability fields can encode an effective κ. Mesa 3's `AgentSet` operations can select, group, and apply functions to arbitrary subsets, including subsets defined by attribute values rather than class. Such selectors permit process-oriented code over overlapping populations, even though class membership and agent records remain the standard presentation of heterogeneous state.

MASON makes the nominal route more explicit through Java classes and interfaces. A model commonly defines several agent classes implementing `Steppable`; class fields determine the usual state schema and the scheduler invokes each object's `step(SimState)` method [4]. Taking the principal concrete class as τ gives the same conventional map from type to Θ. Java interfaces and inheritance can also describe overlapping capabilities, while delegation, composition, and state flags can represent κ without enumerating every capability combination as a subclass. Interfaces are themselves overlapping classifications, so they should not all be conflated with the paper's single, exhaustive τ: scientifically meaningful interface incidence is closer to a component signature. MASON can schedule shared `Steppable` processes or maintain custom

collections, but its native object and scheduler APIs do not turn those capability sets into ECS-style schema queries.

NetLogo separates nominal kinds from classes in the host-language sense. Every individual is first a turtle, patch, or link; `breed` declarations then partition turtles or links into named agentsets, and breed-specific `-own` declarations together with the common `turtles-own`, `patches-own`, and `links-own` declarations specify available variables [5]. A breed is thus a natural τ, the variables available to it define Θ, and their values form θ. Procedures are not methods owned by a class, and `ask` can target any constructed agentset, so behavior may already be written as a population operation. Moreover, a turtle or link can change breed at runtime. NetLogo is therefore an important counterexample to the assumption that τ_t must be lifetime-invariant: changing breed can also change the breed-specific part of Θ_t. Overlapping capabilities can be encoded with ordinary variables, lists, links, or membership predicates and selected by arbitrary agentsets, but breeds themselves remain exclusive nominal categories rather than independently attachable components.

Agents.jl uses Julia's concrete data types rather than a conventional class hierarchy. A homogeneous model may define one agent struct; heterogeneous models may admit a `Union` of agent types or use `@multiagent` to wrap a closed set of variants [6]. Concrete type or enclosed variant then supplies τ, fields determine Θ, and multiple dispatch can specialize behavior by that kind. This representation is agent-centered, but behavior need not be stored on the agent: `StandardABM` accepts external `agent_step!` and `model_step!` functions, and the latter can perform custom selection and scheduling. Traits, delegated state objects, flags, and predicates can emulate overlapping κ-like capabilities. Nevertheless, the built-in heterogeneous containers are organized around a declared union or closed variant set, not around arbitrary combinations of independently stored component schemas.

FLAME GPU sharpens the distinction because execution efficiency is central to its design. Agent types declare fixed variable schemas, while agent functions are associated with agent states and execution layers and operate over GPU populations [7]. The declared agent type again provides a natural τ and schema Θ; dynamic agent state selects which functions apply, but is not by itself an independently attachable component set. FLAME GPU thus places behavior less squarely inside individual objects than Mesa or MASON while still organizing heterogeneous storage primarily by agent type and state. Across all five frameworks, compositional behavior can be emulated. The narrower contrast is that ECS makes κ, signature-based selection, and changes of structural composition native organizing abstractions, whereas these toolkits primarily expose classes, breeds, closed variants, fields, states, and user-defined subset selectors.

2.5. What this architecture does well

- It closely follows ordinary-language descriptions of autonomous individuals.
- The complete state and behavior of one agent can be inspected in one place.
- Highly individualized cognition or event-driven behavior can be natural to express.
- Mature ABM frameworks provide extensive tooling around this representation.

2.6. Limitations to investigate rather than assume

- Growth of type hierarchies or conditional behavior as capabilities overlap.
- Duplication when several agent types share a process.
- Difficulty separating observation from mutation when simultaneous behavior is required.
- Pointer-heavy or array-of-structures layouts that load irrelevant state during population sweeps.

Required standard of comparison

Use an idiomatic agent-centered implementation, not a deliberately weak inheritance hierarchy. Julia is not a conventional class-based OOP language, so describe the Agents.jl implementation as agent-centered. If the paper makes stronger claims about OOP, add a representative class-based implementation or narrow the terminology.

3. Entity Component Systems as a Modeling Architecture

Existing work has established that ECS can be used to implement ABM engines and can improve parallel performance [8]. HECATE applies ECS concepts to engineering multi-agent systems [9]. This paper builds on that literature but shifts the emphasis from engine feasibility to the consequences of ECS for scientific model construction.

3.1. Entities, components, systems, and resources

- **Entity:** an identifier with no intrinsic data or behavior.
- **Component:** a focused unit of state associated with an entity.
- **Component signature:** the set of components currently associated with an entity.
- **Query:** a selection of all entities possessing a required signature.
- **System:** a transformation over components selected by one or more queries.
- **Resource:** model-level state such as parameters, random-number streams, spatial indexes, or aggregate statistics.
- **Structural change:** the addition or removal of entities or components, normally committed at a controlled boundary.

The component labels in Section 2.2 are only tags. In an ECS implementation, their presence determines which component stores and systems are available and can thereby constrain the admissible agent-level parameters. Let $\mathcal{H}$ be a family of parameter spaces. We express this relationship through the schema map

$$\Phi_t : \mathcal{T} \times \mathcal{P}(\mathcal{U}) \to \mathcal{H}, \quad \Theta_t(i) = \Phi_t(\tau_t(i), \kappa_t(i)). \tag{12}$$

The map is intentionally unrestricted. A component may introduce, remove, or constrain several parameter coordinates; one parameter may govern multiple components or systems; and parameters may instead be stored at the type or model level. In particular, this does not impose a component-wise product decomposition of $\Theta_t(i)$ or make every entry of $\theta_t(i)$ a component. Because Φ_t is time-indexed, the admissible parameter space may also change without a structural change.

Represent a system as

$$S_k = (Q_k, \mathrm{Read}_k, \mathrm{Write}_k, F_k), \tag{13}$$

where Q_k selects participating entities, Read_k and Write_k identify the components or resources read and written, and F_k is the transformation. This representation connects model semantics, dependency analysis, testing, and possible parallel execution.

3.2. Compositional heterogeneity

Using the notation of Section 2.2, let $\mathcal{R}_k \subseteq \mathcal{U}$ be the component set required by system k. Its signature query selects

$$E_k(t) = \{i \in I_t \mid \mathcal{R}_k \subseteq \kappa_t(i)\}. \tag{14}$$

This required-signature condition is the simplest form of Q_k; a query may add value predicates or exclude components. The signature determines eligibility, while the system applies one shared process to the selected population.

Develop the following argument:

- An entity does not need to belong to a named class to participate in a behavior.
- Different systems may select overlapping populations.
- A new combination of components does not necessarily require a new agent type.
- Adding or removing a component changes agent structure and may confer or remove a behavioral capability.
- Heterogeneity becomes a property of component incidence as well as parameter values.

3.2.1. Example component-incidence matrix

Entity	Position	Near-field avoidance	Convention perception	Habit formation
Driver A	✓	✓		
Driver B	✓		✓	✓
Driver C	✓	✓	✓	✓
Driver D	✓			✓

The traffic case study realizes this form directly. Six behavioral mechanisms are represented by independently optional component bundles, so systems select overlapping subsets of drivers without defining a class for every component combination. The signature is fixed during one driver's life in the present model, but entry-draw replacement and evolutionary inheritance can change the population distribution between generations. The case therefore demonstrates structural and evolutionary composition, not within-lifetime component acquisition.

3.3. Thinking in systems rather than agents

The agent-centered question is, "What does this agent do during its step?" The system-centered question is, "What transformation occurs in the model, and which entities possess the state required to participate?"

A system-centered transition can be written as

$$S_k : E_k \times W \to \Delta W. \tag{15}$$

Develop three implications:

1. **Processes become explicit.** Perception, choice, movement, matching, learning, entry, and exit appear as separate executable model processes.
2. **Mechanisms become replaceable.** A modeler can substitute a prediction or learning system while retaining entity state and unrelated processes.
3. **Dependencies become inspectable.** Read and write sets identify which systems must be ordered and which may run independently.

This chapter must address a likely objection: locating behavior outside an entity does not remove scientific agency. Private information, goals, memory, and decisions remain entity-

specific components; a system implements the transition law shared by entities possessing its required component signature.

3.4. Interaction phases and concurrency

Distinguish carefully:

- **Simultaneous model semantics:** agents decide from a common state.
- **Concurrent execution:** computations may overlap without changing the declared semantics.
- **Parallel execution:** concurrent work is distributed across hardware for speed.

The ECS decomposition of an interaction should be illustrated as:

```
observe -> form intentions -> resolve conflicts -> commit actions -> learn
```

Systems with disjoint writes or read-only shared inputs may be evaluated concurrently. Systems that contend over shared resources require an explicit reduction, arbitration, or commitment phase. ECS exposes these dependencies but does not eliminate them. The formal concurrency properties of ECS and the conditions for deterministic execution should be connected to [10].

3.5. Data-oriented engineering consequences

Explain the difference between an array of complete agent records and component-oriented storage. Then motivate, without assuming, the following expected effects:

- Systems load only the component arrays they use.
- Homogeneous loops may improve cache locality and vectorization.
- Queries over archetypes can batch entities with compatible memory layouts.
- Explicit read/write sets can support safe multicore scheduling.
- Structure-of-arrays layouts may map naturally to GPU kernels.

Also state the costs:

- Query and archetype-management overhead.
- Expensive structural changes when entities move between archetypes.
- Synchronization and conflict-resolution costs.
- Possible fragmentation of the conceptual description of an individual agent.
- Poor GPU utilization for branch-heavy or tightly coupled interactions.

4. Positioning in the Literature

4.1. ABM tools and agent-centered design

- Review major ABM toolkits and how they represent agent types, activation, interaction, and model-level processes [2].
- Avoid claiming that existing frameworks cannot support heterogeneity, synchronous behavior, or custom scheduling.
- Identify the more precise contrast: ECS makes component composition and population systems the default organizing abstractions.

4.2. ECS, data-oriented design, and concurrency

- Introduce ECS origins and its emphasis on composition over inheritance.
- Discuss formal work on ECS semantics and deterministic concurrency [10].
- Separate the architectural pattern from particular archetype storage implementations.

4.3. ECS in ABM and multi-agent systems

- Present ECS-based ABM engine work, especially its feasibility and CPU-parallel performance emphasis [8].
- Present HECATE's mapping between ECS and multi-agent engineering [9].
- Review GPU ABM frameworks whose agent functions, execution layers, messages, or kernels already resemble population-level systems.

4.4. Proposed gap

The intended novelty claim is:

"Previous work has established the feasibility and performance potential of ECS-based ABM engines. This paper instead examines ECS as a scientific modeling architecture, focusing on structural heterogeneity through dynamic component composition, the representation of behavior as population-level systems, and the resulting formulation of simultaneous agent interactions."

Literature work still required

Conduct a reproducible search across ABM, individual-based modeling, multi-agent systems, component-based simulation, process-oriented simulation, FLAME/FLAME GPU, and data-oriented scientific computing. Do not use "first" or "previously unexplored" until this search is documented. Add literature on ODD model descriptions, activation regimes, component-based ABM, and GPU conflict resolution.

5. Case Study: The Evolution of a Traffic Convention

5.1. Why this model

Use the traffic-convention model associated with Hodgson and Knudsen as a compact economic ABM in which heterogeneous behavioral dispositions, endogenous habit formation, interaction, and convention emergence are tightly connected [1].

The case study should serve four purposes:

1. Validate that ECS can faithfully express an established model.
2. Make the difference between agent-centered and system-centered organization concrete.
3. Extend the model from parametric to compositional heterogeneity.
4. Demonstrate how the same decomposition supports simultaneous interaction.

5.2. Reference model and extension boundary

The repository contains two deliberately distinct models. `SequentialModel` is the agent-centered reference used to reproduce the one-cell, activation-ordered traffic process associated with Hodgson and Knudsen. `CapabilityModel` is the ECS extension specified below. It retains the reference model's two-lane ring, bounded forward observation, LR lane-choice equation, age-dependent habit, and collision-replacement selection. It changes the timing to staged simultaneous action, permits speeds from one to three cells, and makes behavioral mechanisms independently optional. Convention and SocialHabit are extensions and have no counterpart in the reference model. Results from the two models are therefore a contrast in mechanisms and timing, not a claim of numerical equivalence.

Matched initialization and deterministic tests verify shared setup and local transition rules, but they are not sufficient evidence of replication. A behavioral replication must additionally compare published target patterns over multiple seeds and report every intentional deviation listed in the contrast below.

5.3. Capability-composed synchronous model

The following specification is an implementation-level ODD description of the current `CapabilityModel`. A reimplementation that follows the state variables, equations, action ordering, and schedule below should reproduce its scientific semantics. Exact random streams additionally require Julia's `Xoshiro` generator and the same entity/query iteration order.

5.3.1. Purpose, entities, environment, and scale

The model studies whether a population of boundedly informed drivers forms a common rule of the road and how personal Habit, observed Convention, and SocialHabit formed from traces of successful drivers affect that process. A car is an entity. The road is the discrete periodic lattice

$$G = \{1, 2\} \times \mathbb{Z}_H. \tag{16}$$

The first coordinate is the lane and the second is an element of the cyclic group $\mathbb{Z}_H$. Thus longitudinal addition is explicitly modulo H, whereas the lane coordinate is merely an element of the finite set $\{1, 2\}$ and does not wrap. Every cell contains at most one car at the committed

start of a tick. Direction $d_i \in \{-1, +1\}$ is permanent during a driver's life: $+1$ moves toward increasing longitudinal coordinates (clockwise), and -1 moves toward decreasing coordinates (counterclockwise). Define the relative-side sign

$$r(x, d) = \begin{cases} +1 \text{ when lane } \ x \ \text{ is left relative to direction } \ d \\ -1 \text{ otherwise} \end{cases}. \tag{17}$$

Thus $r(1, +1) = r(2, -1) = +1$ and $r(2, +1) = r(1, -1) = -1$. One tick contains three movement micro-steps. A car may advance between one and $M \leq 3$ cells per tick and may change lane only during its first micro-step. The population size is held constant by replacing every driver removed in a collision.

5.3.2. Entity state and structural capabilities

Every capability car has the following mandatory state.

State	Domain	Meaning
Position	G	Committed lane and longitudinal cell
PrevPosition	G	Position copied at the beginning of the tick
Direction	$\{-1, +1\}$	Permanent direction of travel
Speed	$\{1, \ldots, M\}$	Last successfully committed speed
SpeedAdjustment	positive integer	Per-driver speed cap, set to the global M
Step	positive integer	Successful lifetime counter; initialized to one
LocalObservation	record	Private start-of-tick observation summary
LaneScore , LR	real	Unperturbed lane-response value
LaneProposal	$\{1, 2\}$	Lane selected from LR
SpeedProposal	$\{1, \ldots, M\}$	Submitted speed
MovementPath	G^3	Three-micro-step proposed path

Six optional components determine which response systems select a car. Their presence indicators are written $\chi_i^S, \chi_i^O, \chi_i^A, \chi_i^H, \chi_i^C, \chi_i^Z$.

Capability component	Private state or trait	Information used
SameDirectionResponse	s_i	Sides occupied by observed co-directional cars
OppositeDirectionResponse	o_i	Sides occupied by observed opposing cars
NearFieldAvoidance	a_i	Counts on both relative sides within M cells
HabitFormation	h_i and acquired H_i	Driver's own realized lane history
ConventionPerception	$\alpha_i^C, \sigma_i^C, C_i, \rho_i$	History of nearby drivers' realized side choices

Capability component	Private state or trait	Information used
SocialHabitFormation	$z_i, \alpha_i^Z, \sigma_i^Z, Z_i$	History of locally observed successful-driver traces

Habitus, PerceivedConvention, and SocialHabitus exist only when their corresponding formation/perception component exists. Every car has speed control; speed is not an optional behavioral component in this treatment. With six independent binary capabilities, up to 2^6 component signatures can occur without defining combination-specific driver types.

5.3.3. Parameters and initialization

Parameter	Default	Definition
N, H	$120, 300$	Population and longitudinal ring length
ℓ	60	Forward observation horizon; current experiments set $\ell = 20$
δ	0.2	Standard deviation of entry trait draws around one
ε	0.01	Probability of reversing the LR-selected relative side
K	10	Habit accumulation offset
M	3	Maximum speed and near-field distance
w_S, w_O, w_A	0.5	Traffic-response weights
w_H, w_C, w_Z	0.5	Habit, Convention, and SocialHabit weights
q_S, q_O, q_A	0.75	Independent entry probabilities for traffic responses
q_H, q_C, q_Z	$0.5, 0.5, 0$	Independent entry probabilities for acquired mechanisms
α^C, α^Z	0.2	Entry learning rates
σ^C, σ^Z	0.05	Entry observation-noise standard deviations
ρ, D	$0.9, 0.25$	Trace retention and deposit magnitude
γ	0	Optional speed-dependent clearance coefficient
μ, σ_m	$0.02, 0.05$	Evolutionary presence-mutation probability and trait-mutation scale

The six presence indicators are drawn independently with probabilities q_k. Conditional on presence, s_i, o_i, a_i, h_i, and z_i are independent draws from $\mathcal{N}(1, \delta^2)$. Convention carriers receive the common entry values α^C, σ^C; SocialHabit carriers receive z_i plus α^Z, σ^Z. Acquired states start at $H_i = C_i = \rho_i = Z_i = 0$.

Initial positions are sampled uniformly without replacement from the $2H$ cells. Direction labels are shuffled after assigning equal counts clockwise and counterclockwise (for an odd population, clockwise receives the extra car). Initial speeds are independent discrete-uniform draws from 1 through M. Step is initialized to one. The implementation rejects $N > 2H$, requires exactly two lanes, and requires $H \geq 2M + 1$.

The reported capability experiments override the entry probabilities according to treatment. All three traffic-response capabilities are present ($q_S = q_O = q_A = 1$). A pure Habit, Conven-

tion, or SocialHabit treatment sets the corresponding acquired capability probability to one and the other two to zero. Mixture treatments set $q_H = q_C = q_Z = 0.5$. They use 5,000 ticks, discard the first 1,000, set $\ell = 20$, and retain the other values above.

5.3.4. Bounded local observation

At the beginning of tick t, the model rebuilds a two-dimensional occupancy array from committed positions. Driver i observes both lanes at forward distances $k = 1, \ldots, \min(\ell, H - 1)$, where forward means adding kd_i modulo H. The driver's current longitudinal row ($k = 0$) is excluded. Other drivers' traits, memories, LR values, and current proposals are never observable.

For co-directional cars let n_i^S be the number observed and n_i^{SL} the number on the left relative to i's direction. Define

$$\mathrm{SL}_i = \begin{cases} 0.5 \text{ if } \; n_i^S = 0 \\ \frac{n_i^{\mathrm{SL}}}{n_i^S} \text{ otherwise} \end{cases}. \tag{18}$$

Define OL_i analogously for opposing cars. Its left/right classification is also relative to the observing driver. Let CL_i and CR_i be counts of all cars, irrespective of direction, observed on the relative left and right at distances $k \le M$. These are counts rather than proportions.

Convention observes what side other drivers selected in their own frame of reference. If O_i is the set of observed cars, the current convention sample is

$$|c|_i = \frac{1}{|O_i|} \sum_{j \in O_i} r(x_j, d_j). \tag{19}$$

It is undefined when O_i is empty. Notice the distinction: SL_i and OL_i classify lanes relative to the observer, whereas $|c|_i$ evaluates each observed car relative to that car's own direction.

SocialHabit does not observe cars' success directly. The environment stores a signed trace field $T_{t(x,y)} \in [-1, 1]$. The observation window samples every cell whose absolute trace is at least 10^{-8}, whether or not it is currently occupied, and computes their arithmetic mean $|T|_i$. No sample is produced when the window contains no trace above the cutoff.

5.3.5. Convention and SocialHabit learning

Learning occurs after observation and before the current LR is calculated, so both mechanisms use committed positions and the trace field left by earlier ticks. For a Convention carrier with at least one observed car, draw the standard-normal variate ξ_i^C and calculate

$$\tilde{c}_i = \mathrm{clip}\big(|c|_i + \sigma_i^C \xi_i^C, -1, 1\big), \tag{20}$$

$$C_{i'} = \mathrm{clip}\big((1 - \alpha_i^C) C_i + \alpha_i^C \tilde{c}_i, -1, 1\big), \tag{21}$$

$$\rho_{i'} = \mathrm{clip}\big((1 - \alpha_i^C) \rho_i + \alpha_i^C, 0, 1\big). \tag{22}$$

If no car is observed, both C_i and confidence ρ_i remain unchanged. For a SocialHabit carrier with at least one trace sample, independently draw the standard-normal variate ξ_i^Z and update

$$\tilde{T}_i = \mathrm{clip}\big(|T|_i + \sigma_i^Z \xi_i^Z, -1, 1\big), \tag{23}$$

$$Z_{i'} = \mathrm{clip}\big((1-\alpha_i^Z) Z_i + \alpha_i^Z \tilde{T}_i, -1, 1\big). \tag{24}$$

Without a trace sample, Z_i is unchanged. SocialHabit has no separate confidence state. Convention therefore builds a history over other drivers' realized side choices; SocialHabit builds a history over an environmental, vanishing record of successful movement.

5.3.6. Lane-response equation

After learning, every lane score is reset to zero. A system contributes to a driver's score only when the corresponding component is present. Separate the traffic-response and acquired-disposition terms as

$$B_i = \chi_i^S w_S s_i (2\mathrm{SL}_i - 1) - \chi_i^O w_O o_i (2\mathrm{OL}_i - 1) + \chi_i^A w_A a_i (\mathrm{CR}_i - \mathrm{CL}_i), \tag{25}$$

$$A_i = \chi_i^H w_H h_i H_i + \chi_i^C w_C \rho_i C_i + \chi_i^Z w_Z z_i Z_i, \tag{26}$$

and set $\mathrm{LR}_i = B_i + A_i$.

A positive score selects left relative to travel direction and a negative score selects right. An exact zero preserves the driver's currently realized relative side. After this deterministic choice, the side is reversed with probability ε. Clockwise drivers map relative left/right to absolute lanes one/two; counterclockwise drivers map them to lanes two/one. The stored `LR` is the unperturbed score, not the error-flipped action.

The three acquired capabilities thus have the same downstream function but different information sources: each adds one term to LR. Habit is personal lane history, Convention is local history of other drivers' choices, and SocialHabit is local history over decaying success traces.

5.3.7. Speed choice and private path construction

For candidate lane $\lambda \in \{1, 2\}$ and speed $v \in \{1, \ldots, M\}$, construct a three-position path. At micro-step one the car moves one longitudinal cell and enters λ; at each subsequent micro-step up to v it advances one more cell in the same lane; after micro-step v it remains at its final position.

The safety screen is bounded in what it predicts. Driver i extrapolates every other car j by assuming that j keeps its currently committed lane and current speed. It does not inspect j's lane or speed proposal. The current implementation loops over all other cars rather than applying the observation horizon, although with zero clearance only cars whose three-step paths can intersect can reject the candidate. A candidate is rejected if, at any aligned micro-step, its path and an extrapolated path:

- occupy the same cell;
- exchange cells across an edge; or
- start on the same longitudinal row and cross diagonally while exchanging lanes.

If $\gamma > 0$, define the integer clearance $g(v) = \lceil \gamma(v-1) \rceil$. A candidate is additionally rejected whenever both paths occupy the same lane at an aligned micro-step and their minimum

circular longitudinal separation is at most $g(v)$. Clearance depends on the focal candidate's speed and is zero under the default treatment.

The default action search is lexicographic and speed-first:

```
for speed = M, M-1, ..., 1
  for lane = (LR-selected lane, other lane)
    choose and stop at the first action passing the safety screen
```

The optional lane-first treatment reverses the loop nesting: it exhausts speeds on the LR-selected lane before considering the other lane. If no candidate passes, the submitted fallback is speed one on the LR-selected lane even though it is not certified safe. Screening cannot guarantee collision-free movement: all drivers screen against lagged observable motion, while actual proposals are formed simultaneously.

5.3.8. Synchronous conflict resolution

After every proposal is fixed, conflict resolution uses the submitted paths, not the extrapolations used by the safety screen. Cars are ordered by stable entity identifier and marked active. For micro-steps $m = 1, 2, 3$:

1. Determine position p_i^m for every currently active car.
2. Mark every car sharing a cell with another active car.
3. Mark both cars in every exact edge swap or diagonal lane crossing between $p_i^{\{m-1\}}$ and p_i^m.
4. Remove all newly marked cars from the active set before the next micro-step.

All marked cars die; there is no winner for a contested cell. A car killed at an early micro-step cannot cause a later conflict. Survivors commit their final path position, adopt the proposed speed, and increment `Step` by one. Killed entities are removed only after their travel directions have been recorded for replacement.

5.3.9. Successful-driver traces and personal Habit

Trace updating occurs after failed drivers are removed and before replacements enter. First multiply the complete trace field by retention ρ and set values with magnitude below 10^{-8} to zero. Every surviving driver then deposits $Dr(x_i, d_i)$ at every cell it actually traversed, once per completed micro-step up to its committed speed. Each addition is clipped to $[-1, 1]$. Failed drivers leave no trace, stopped path padding leaves no trace, and newborns leave no trace on entry.

After replacement, Habit carriers that survived the tick update their personal state. Let a_i be `Step` after its successful increment. Then

$$H_{i'} = \mathrm{clip}\left(H_i + \frac{r(x_i, d_i)}{K + a_i}, -1, 1\right). \tag{27}$$

Newborns have `Step` equal to one and skip this update. Consequently Habit is the Hodgson–Knudsen age-dependent disposition generated only by the driver's own realized side and successful lifetime. Convention and SocialHabit are not updated by this equation.

5.3.10. Collision replacement and evolution

Every killed driver produces one replacement request carrying only its travel direction. Replacements are shuffled and placed uniformly without replacement among cells left empty after conflict resolution. Their initial speed is drawn uniformly from $1, \ldots, M$, their acquired states are zero, and `Step` is one.

Under `EntryDrawReplacement`, each optional capability is redrawn independently with its entry probability q_k. Present response/disposition traits s_i, o_i, a_i, h_i, z_i receive fresh $\mathcal{N}(1, \delta^2)$ draws; Convention and SocialHabit learning/noise parameters receive their configured entry values. This is the default and keeps entry composition exogenous.

Under `EvolutionaryReplacement`, the parent pool is the set of survivors before any newborn is created. Each newborn selects one parent uniformly with replacement. For every optional capability, presence is toggled independently with probability μ: a present capability is lost, while an absent capability is gained only when its configured entry share is nonzero. If presence is not toggled and the capability is present, each continuous trait receives an independent $\mathcal{N}(0, \sigma_m^2)$ perturbation. Nonnegative traits are truncated at zero; learning rates are clipped to $[0, 1]$. An empty survivor pool falls back to an entry draw. The newborn inherits neither H_i, C_i, ρ_i, nor Z_i. Evolution therefore transmits capability structure and stable traits, not acquired experience. In both replacement regimes, the newborn takes the direction of the driver whose collision created the request, so directional population counts remain fixed.

5.3.11. Complete tick schedule and system dependencies

The exact update schedule is:

1. copy committed positions to previous-position state
2. rebuild occupancy from committed positions
3. observe traffic and the previous trace field
4. update Convention and SocialHabit memories
5. reset and sum capability-specific LR contributions
6. select the relative side, apply decision error, and propose a lane
7. screen lane-speed actions and store three-micro-step paths
8. resolve submitted paths synchronously and remove failed drivers
9. decay the trace field and deposit traces from surviving paths
10. create entry-draw or evolutionary replacements
11. update personal Habit for surviving Habit carriers
12. rebuild occupancy, update aggregates, and log the committed state

Phase	Principal reads	Writes or structural effects
Snapshot and observation	Positions, directions, occupancy, traces, horizon	`PrevPosition`, occupancy, `LocalObservation`
Learning and LR	Observations, optional traits, acquired states	C_i, ρ_i, Z_i, score, `LR`, lane proposal
Speed proposal	Current motion, lane proposal, speed policy	Speed proposal and private path
Conflict resolution	All submitted paths	Survivor positions/speeds/ages; remove failed entities

Phase	Principal reads	Writes or structural effects
Trace update	Surviving paths and directions	Environmental trace resource
Replacement	Requests, free cells, entry shares or survivor genomes	Create newborn entities and component signatures
Habit and logging	Committed survivor sides and ages	H_i, occupancy, aggregate time series

The two occupancy rebuilds are semantically distinct. The first freezes the common information state used for decisions; the second makes the post-movement, post-replacement state available to diagnostics and the next tick. Convention and SocialHabit learning must precede current proposals, whereas personal Habit must follow realized movement.

5.3.12. Recorded outcomes

The principal convention statistic is

$$Q_t = \left| \frac{1}{N} \sum_{i=1}^{N} r(x_i(t), d_i) \right|. \tag{28}$$

$Q_t = 1$ means all drivers use the same relative side and $Q_t = 0$ is a balanced population. Experiments additionally report the fraction of post-burn-in ticks with $Q_t \geq 0.8$, first passage to that threshold, coordinated-episode lengths, mean speed, replacement count per car-step, acquired-disposition magnitudes, final marginal shares of the optional mechanism components, and the effective number of structural component profiles

$$D_t = \exp\left(- \sum_{x \in \{0,1\}^6} \pi_t(x) \log(\pi_t(x)) \right), \tag{29}$$

Here π_t is evaluated over the six optional mechanism components. Their presence uniquely determines the associated optional state components in this model; the convention $0 \log(0) = 0$ is used. A replacement in the logger is identified by `Step == 1`, which is equivalent to a collision death because population size is constant and every death is replaced within the same tick.

5.4. Contrast with the sequential reference

Feature	Sequential reference	Capability model
Software organization	One `Car` record contains state and all traits	Entity plus mandatory and optional components selected by systems
Decision timing	Stable-ID activation; later cars observe earlier movements	All cars observe one committed state and submit before resolution
Movement	Exactly one longitudinal cell per tick	One to three cells in three synchronous micro-steps

Feature	Sequential reference	Capability model
Observation window	Includes the current row and looks forward ℓ cells	Excludes current row and looks forward ℓ cells
Near field	Distances zero through two	Distances one through M
Lane equation	Same, opposite, avoidance, and optional Habit terms	Same core terms plus independently optional Habit, Convention, and SocialHabit
Safety choice	No pre-screen; move directly to intended one-step cell	Current-motion path screen before simultaneous proposals are resolved
Collision	Same destination, swap, or diagonal crossing after one-cell movement	Same conflicts at every movement micro-step
Habit	Every survivor updates own-side habit; w_H controls its effect on LR	Update applies only to entities carrying `HabitFormation`
Replacement	Fresh four-trait entry draw	Independent entry draw or survivor inheritance with mutation

The repository also contains a simultaneous activation treatment of the agent-centered reference, in which all one-step intentions are calculated from a frozen state before commitment. It isolates activation timing while retaining the reference agent representation. It should not be conflated with `CapabilityModel`, which simultaneously changes component composition, speed, observation, safety screening, collision resolution, and replacement options.

5.5. Forecast-and-control extension

Use the existing occupancy-strategy analysis as evidence that separating prediction from choice makes behavioral mechanisms independently substitutable. Focus the main text on:

- Naive current-lane prediction.
- Two-frame temporal-consistency prediction.
- Iterated decision-aware prediction.

The remaining strategies can be a compact robustness table or appendix. Emphasize that a predicted state affects decisions and therefore changes the state being predicted: these are forecast-and-control policies, not passive forecasts.

6. Comparative Evaluation

6.1. Evaluation logic

The paper cannot prove an architectural advantage from one attractive code example. Combine a design comparison, behavioral experiments, and performance measurement.

Claim	Evidence	Threat to address
Compositional heterogeneity	Add and remove capabilities without defining combination-specific types	Idiomatic OOP can also use composition
System-centered modularity	Substitute mechanisms while sharing state and unrelated processes	Lines of code alone are a weak measure
Natural simultaneous interaction	Derive staged formulation from system dependencies	Other ABM frameworks can also use buffers and phases
Parallel scalability	Thread, memory, and possibly GPU benchmarks	Interaction conflicts may dominate

6.2. Experiment A: Representing heterogeneity

- Define a set of independent behavioral capabilities.
- Construct populations with overlapping component signatures.
- Add one new capability after both implementations are complete.
- Compare the required changes to state definitions, behavior dispatch, initialization, logging, and analysis.
- Report qualitative dependency changes and limited quantitative measures such as touched modules, duplicated transition logic, and combinations represented.

The conclusion should concern locality and composability, not the impossibility of implementing the same model with objects.

6.3. Experiment B: Substituting systems

- Hold entities and unrelated processes constant.
- Exchange prediction, learning, or conflict-resolution systems.
- Verify that unaffected mechanisms are reused without conditional branches.
- Use the existing strategy results to illustrate scientific experiments enabled by this separation.

6.4. Experiment C: Interaction semantics

Compare fixed sequential, shuffled sequential, synchronous, and staged interaction only where they are scientifically meaningful. Hold behavioral equations and initial conditions constant. Measure:

- Convention strength and persistence.
- Time to convention.
- Collision events and removed agents.

- Throughput or successful movements.
- Sensitivity to density and capability composition.

This experiment supports the interaction argument; it should not displace the broader architectural contribution.

6.5. Experiment D: Engineering performance

Benchmark at least:

1. Agent-centered serial implementation.
2. ECS serial implementation.
3. ECS threaded implementation.
4. ECS GPU implementation, only if it is complete enough for a fair comparison.

Report:

- Wall-clock time after compilation and warm-up.
- Population and component-count scaling.
- Allocations and peak memory.
- Cache misses and memory bandwidth where measurable.
- Thread scaling and parallel efficiency.
- Time spent in queries, systems, structural changes, synchronization, and conflict resolution.
- Hardware, software versions, compiler settings, and number of repetitions.

Use both compute-light and interaction-heavy workloads. A synthetic component-sweep benchmark can isolate memory layout, but the traffic model must show end-to-end behavior.

6.6. Randomness, inference, and reproducibility

- Pre-generate or index stochastic draws by replicate, time, entity, and event so architectural treatments do not merely consume different RNG streams.
- Distinguish matched initialization from genuinely paired subsequent shocks.
- Predefine primary contrasts and outcome measures.
- Report Monte Carlo uncertainty and effect sizes.
- Use longer horizons and multiple initial conditions.
- Archive code, project manifests, raw replicate data, and exact reproduction commands.

7. Results

Do not turn this section into prose yet

Organize results by research question, not by the order in which scripts were written. Each subsection should begin with a one-sentence answer, then show the evidence and its uncertainty.

7.1. RQ1: Compositional heterogeneity

- Report the number and distribution of component signatures used in the experiment.
- Show how a capability is introduced or removed in both architectures.
- Report code-locality or dependency evidence without treating code size as scientific proof.
- Analyze whether structural heterogeneity changes convention formation beyond parametric heterogeneity.

7.2. RQ2: System-centered model organization

- Present the system dependency graph.
- Show which systems are reused across behavioral variants.
- Report unit and integration tests at system boundaries.
- Discuss whether the executable structure corresponds more directly to the published process description.

7.3. RQ3: Simultaneous and parallel interactions

- Compare sequential and staged semantics.
- Quantify update-order sensitivity.
- Show whether thread count changes results under a fixed declared semantics.
- Report conflict frequency and resolution cost.

7.4. RQ4: Engineering performance

- Separate data-layout gains from threading gains.
- Show scaling curves rather than a single population size.
- Identify cross-over points where ECS overhead becomes worthwhile.
- Report workloads where ECS provides little or no advantage.

7.5. Supporting forecast-strategy results

Current exploratory findings to preserve and later verify:

- Decision-aware prediction produces the strongest survival and replacement outcomes among the tested policies.
- Two-frame Naive provides a simpler improvement using only current-time information.
- The ranking is robust across the tested densities and much of the behavioral-weight grid.
- These findings demonstrate mechanism substitution and coupled forecast-and-control, not by themselves the superiority of ECS.

8. Discussion

8.1. The agent without an agent object

Address whether ECS undermines autonomy. Proposed position:

- An agent is a scientific unit with identity, state, information, and possible actions.
- A software object is only one implementation of that unit.
- ECS externalizes shared transition laws while retaining entity-specific state.
- Agency can therefore remain meaningful without behavior being stored in an object method.

8.2. What ECS changes for modelers

- Model construction begins with processes and required state rather than a complete taxonomy of actors.
- Heterogeneity can be expressed through overlapping capabilities.
- Model mechanisms can be tested and substituted as systems.
- Timing, conflicts, and dependencies must be made explicit.
- The same explicitness can support deterministic concurrency and efficient batch execution.

8.3. What ECS does not solve

- It does not determine the scientifically correct timing semantics.
- It does not make conflicting interactions automatically parallel.
- It does not guarantee cache or GPU performance.
- It does not remove the need for validation, calibration, or sensitivity analysis.
- It may make an individual agent's complete behavior harder to inspect.
- It can be excessive for small models with a single stable agent type and strongly individualized logic.

8.4. Implications for economic ABMs

Develop examples beyond traffic:

- A firm can acquire exporter, borrower, employer, or innovator capabilities without becoming a new nominal type for every combination.
- A household can participate in labor, credit, housing, and consumption systems according to its current components.
- Institutions can be modeled as systems or resources rather than necessarily as agents, forcing the modeler to state where agency is substantively intended.
- Entry, bankruptcy, learning, and institutional change can be represented as transformations of component composition.

These examples should remain implications unless they are implemented as additional case studies.

8.5. Threats to validity

- One traffic model cannot establish universal architectural superiority.

- The agent-centered implementation may reflect author familiarity or framework-specific constraints.
- The chosen ECS library may conflate the abstract pattern with a particular storage implementation.
- Performance results may be hardware- and workload-specific.
- Dynamic composition can introduce semantic choices about when component changes take effect.
- The distinction between an entity, an agent, and an environmental object must be documented explicitly.

9. Conclusion

Restate the intended conclusion cautiously:

Entity Component Systems should be evaluated in ABM not only as a performance technique but as an alternative architecture for model specification. Their main scientific promise lies in representing heterogeneous agents as changing compositions of capabilities, organizing behavior as explicit population-level processes, and exposing the dependencies involved in simultaneous interactions. Cache-efficient storage and parallel execution are important consequences, but their benefits remain empirical and workload-dependent.

The final paragraph should identify the next research step: application to a larger economic model with multiple institutional roles and endogenous changes in agent capabilities.

10. Appendix Roadmap

10.1. Complete ODD description

- Purpose and patterns.
- Entities, state variables, and scales.
- Process overview and scheduling.
- Design concepts.
- Initialization.
- Input data.
- Submodels and equations.

10.2. Architecture comparison

- Full agent-centered pseudocode.
- Full ECS system table with queries, reads, writes, and structural changes.
- Dependency graph and execution phases.
- Definition of every architecture-specific deviation.

10.3. Parameters and experimental design

- Default parameters and scientific interpretation.
- Capability-composition treatments.
- Density, horizon, and sensitivity grids.
- Replicate counts and seed construction.
- Primary and secondary outcomes.

10.4. Additional results

- Full forecast-strategy tables.
- Weight sensitivity.
- Longer-horizon and initialization robustness.
- Convergence diagnostics for iterated decision-aware prediction.
- Complete performance profiles.

10.5. Reproducibility

- Repository and archived release.
- Julia, Rust, Typst, and package versions.
- Hardware and operating-system description.
- Commands for tests, simulations, figures, benchmarks, and paper compilation.

Bibliography

[1] G. M. Hodgson and T. Knudsen, *Economics in the Shadows of Darwin and Marx: Essays on Institutional and Evolutionary Themes*. Edward Elgar Publishing, 2006. doi: 10.4337/9781781007563°.

[2] S. Abar, G. K. Theodoropoulos, P. Lemarinier, and G. M. P. O'Hare, "Agent Based Modelling and Simulation Tools: A Review of the State-of-Art Software," *Computer Science Review*, vol. 24, pp. 13–33, May 2017, doi: 10.1016/j.cosrev.2017.03.001°.

[3] E. ter Hoeven *et al.*, "Mesa 3: Agent-Based Modeling with Python in 2025," *Journal of Open Source Software*, vol. 10, no. 107, p. 7668, 2025, doi: 10.21105/joss.07668°.

[4] S. Luke, C. Cioffi-Revilla, L. Panait, K. Sullivan, and G. Balan, "MASON: A Multiagent Simulation Environment," *SIMULATION*, vol. 81, no. 7, pp. 517–527, 2005, doi: 10.1177/0037549705058073°.

[5] U. Wilensky, "NetLogo." Accessed: Aug. 12, 2026. [Online]. Available: http://ccl.northwestern.edu/netlogo/°

[6] G. Datseris, A. R. Vahdati, and T. C. DuBois, "Agents.jl: A Performant and Feature-Full Agent-Based Modeling Software of Minimal Code Complexity," *SIMULATION*, vol. 100, no. 10, pp. 1019–1031, 2024, doi: 10.1177/00375497211068820°.

[7] P. Richmond, R. Chisholm, P. Heywood, M. K. Chimeh, and M. Leach, "FLAME GPU 2: A Framework for Flexible and Performant Agent-Based Simulation on GPUs," *Software: Practice and Experience*, vol. 53, no. 8, pp. 1659–1680, 2023, doi: 10.1002/spe.3207°.

[8] A. Ambrosio, D. D. Vinco, F. Foglia, C. Spagnuolo, and V. Scarano, "The Impact of ECS Logic on Parallel Performance in Agent-Based Model Simulations."

[9] A. Casals and A. A. F. Brandão, "HECATE: An ECS-based Framework for Teaching and Developing Multi-Agent Systems." Accessed: Feb. 02, 2026. [Online]. Available: http://arxiv.org/abs/2509.06431°

[10] P. Redmond, J. Castello, J. M. Calderón Trilla, and L. Kuper, "Exploring the Theory and Practice of Concurrency in the Entity-Component-System Pattern," *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA2, pp. 1–28, Oct. 2025, doi: 10.1145/3763050°.